

# AlDa-Klausur 1

February 2, 2026

## Aufgabe 1: O-Kalkül

### Aufgabe 1.a: O-Notation

(1)

$$O\left(\frac{\log n}{\log \log n}\right) \subset O((\log n)^2) \subset O(\sqrt{n^{1.5}} + 1000 \cdot n^{1.2}) \subset O(2^{\sqrt{n}})$$

(2)

### Aufgabe 1.b: Laufzeitabschätzung

(1)

int

$$k = 0 \rightarrow O(1)$$

outer loop: for i in 1..n, double i

$$\rightarrow \frac{n}{2} \rightarrow O(\log n)$$

inner loop: for j in n..i, sub j 1

$$\rightarrow i := n, j := i \rightarrow O(n)$$

Gesamt: outer loop \* inner loop:

$$O(n \log n)$$

(2)

outer loop: for i in 0..n, add i 2 = $\frac{n}{2}$  Steps = $\frac{n}{2}$  inner loop: for j in n..1, sub j 1 = $\frac{n}{2}$  Gesamt:

$$O(n^2)$$

(3)

Queue FIFO:  $n$  for  $v$  in  $a$   $\Rightarrow O(n)$  while not queue  $j$ -empty()?  $\Rightarrow$  worst Case  $O(n)$  while-nested for loop: for  $w : v$   $j$ -succ  $\Rightarrow$  if  $:$   $O(1)$ , for  $:$   $O(n)$   
Gesamt:

$$O(n) + O(n^2) \rightarrow O(n^2)$$

## Aufgabe 2: Teile und Herrsche

### Aufgabe 2.a: Laufzeit von Teile und Herrsche

(1)

int  $d \dots \Rightarrow O(1)$  if  $d == (0 \text{ or } 1)? \Rightarrow O(1)$  int  $m \dots \Rightarrow O(1)$  for  $i$  in left.. $m$ , add  $i+1 \Rightarrow O(m) \Rightarrow O(n/2)$  return ...  $\Rightarrow$  Watershed-Konstante mit Master-Theorem  $\Rightarrow$  Watershed  $:= 1 \Rightarrow$

$$n^1$$

$\Rightarrow O(n)$  Gesamt:  $O(3) \Rightarrow O(1) \Rightarrow O(1) + O(n/2) + O(\log n) \Rightarrow O(n \log n)$

(2)

if ...  $\Rightarrow O(1)$  m

$$\rightarrow O(1)$$

$h(\dots); h(\dots); \Rightarrow$  Watershed und Master-Theorem

$$\rightarrow \Theta(n)$$

### Aufgabe 2.b: Binärdarstellung

(1)

$$\text{binary}(x \in \mathbb{N}) :=$$

$n < 2 ?$  ret  $n$   
else ret  $\text{binary}(\text{div } n \ 2) ++ \text{print mod } n \ 2$

(2)

Master-Theorem:  $O(\log n)$

## Aufgabe 2.c: Anzahl von Vertauschungen

(1)

```
dac_mergesort(a :: [n ∈ ℤ], l, m, r ∈ ℤ) :=  
  i, j, k ∈ ℤ, aux :: [x ∈ ℤ]  
  for i in m+1..1, sub i 1 → aux (i-1) := a (i-1)  
  for j in m..r, add j 1 → aux (r+m-j) := a (j+1)  
  n ← 0  
  for k in 1..r, add k 1  
    if aux (j) < aux (i) → a (k) := aux (j); sub j 1;  
    if i ≤ m → n ← add (n (m-i+1))  
    else → a (k) := aux (i); add i 1  
  ret n
```

```
dac_count(a :: [n ∈ ℤ], l, r ∈ ℤ) :=  
  if r ≤ 1 → ret Nil  
  else  
    m ← div (add (1 r) 2)  
    dac_count(a, 1, m)  
    dac_count(a, m+1, r)  
    dac_mergesort(a, 1, m, r)
```

(2)

mergesort first loop: Copy-Aufruf:  $O(n)$   
mergesort second loop: Copy-Aufruf:  $O(n)$   
→  $O(n)$   
third loop:  $O(n)$   
mergesort ::  $O(n)$   
count :: Master-Theorem ::  $O(\log n)$   
Gesamt:  $O(n \log n)$

## Aufgabe 3: Hashen

Aufgabe 3.a: Multiplikative Methode

## Aufgabe 4: Sortieren

Aufgabe 4.a: Mergesort

50, 51, 2, 9, -2, 17